

MPCArbiter: Detect Numeric Error Vulnerabilities in MPC Implementations via Distribution-Aware Differential Oracle

Guopeng Lin
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
17302010022@fudan.edu.cn

Zhengting Jin
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
25213050022@m.fudan.edu.cn

Shuyu Chen
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
23110240005@m.fudan.edu.cn

Jingwei Pu
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
25213050311@m.fudan.edu.cn

Zheng Qu
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
24110240070@m.fudan.edu.cn

Jiaheng Zhang
National University of Singapore
Singapore, Singapore
jhzhhang@nus.edu.sg

Weili Han*
Laboratory for Data Security and
Governance, Fudan University
Shanghai, China
wlhan@fudan.edu.cn

Abstract

Driven by emerging privacy regulations, multi-party computation (MPC) protocols have been widely adopted for cross-organizational data analysis with privacy preservation. Nevertheless, due to the inherent complexity of MPC protocols, MPC implementations often contain numeric error vulnerabilities (NEVs), which may yield incorrect results in data analysis tasks—e.g., misclassifying a malignant tumor as benign. Therefore, detecting and mitigating NEVs is crucial for the practical deployment of MPC protocols. However, since most MPC implementations rely on probabilistic truncation protocols to handle real-number computation, detecting NEVs faces a fundamental challenge: when we observe a discrepancy between the output of an MPC implementation and the corresponding real-number result, it is difficult to determine whether the discrepancy stems from NEVs or just from random truncation noise, which is introduced by probabilistic truncation protocols.

In this paper, we propose MPCArbiter, a practical framework for detecting NEVs by establishing a distribution-aware differential oracle. Our key insight is that while truncation noise varies randomly across executions, correct implementations of the same MPC protocol with the same input should produce statistically identical output distributions. Based on this insight, our proposed oracle compares output distributions across different MPC implementations of the same protocol with the two-sample Kolmogorov-Smirnov test, reliably distinguishing NEVs from truncation noise. Besides, to reduce debugging effort, we complement MPCArbiter with an

NEV localization method based on MPC operation dependency graph. Evaluating MPCArbiter on 36 MPC implementations of four mainstream MPC frameworks (CrypTen, TF-Encrypted, MP-SPDZ, SecretFlow), we discover 10 unique NEVs. Notably, we reveal an NEV in probabilistic truncation protocols that has been widely overlooked by the MPC community. This NEV persists in multiple influential research papers and has propagated into multiple MPC frameworks.

CCS Concepts

• **Security and privacy** → *Software security engineering*; **Privacy-preserving protocols**.

Keywords

Multi-party Computation; Vulnerability Detection; Differential Testing

ACM Reference Format:

Guopeng Lin, Zhengting Jin, Shuyu Chen, Jingwei Pu, Zheng Qu, Jiaheng Zhang, and Weili Han. 2026. MPCArbiter: Detect Numeric Error Vulnerabilities in MPC Implementations via Distribution-Aware Differential Oracle. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26)*, November 15–19, 2026, The Hague, Netherlands. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3830454.3832686>

1 Introduction

Due to the emerging privacy-protection laws and regulations in recent years, e.g., the EU's GDPR [30], multi-party computation (MPC) protocols have been widely adopted by companies and institutions. As a key technology for privacy preservation, MPC protocols enable multiple mutually distrusting parties to perform various joint data analysis tasks, e.g., medical diagnosis [16] and financial risk assessment [3], while keeping their data private. This capability

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *CCS '26, The Hague, Netherlands*.

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
<https://doi.org/10.1145/3830454.3832686>

helps resolve the conflict between data utility and data privacy, positioning MPC protocols as a critical enabler for cross-organizational data analysis.

Nevertheless, due to the inherent complexity of MPC protocols, their implementations often contain numeric error vulnerabilities (NEVs), which can lead to incorrect results in data analysis tasks and are usually difficult for developers to discover. As a concrete example, our evaluation discovers an NEV in SecretFlow [29]: when truncating f bits from additive secret shares, the shares $\llbracket x \rrbracket_1$ and $\llbracket x \rrbracket_2$ should be truncated asymmetrically ($\llbracket x \rrbracket_1 \gg f$ and $-(\llbracket x \rrbracket_2) \gg f$) to keep the result unbiased, but SecretFlow truncates both shares symmetrically ($\llbracket x \rrbracket_1 \gg f$ and $\llbracket x \rrbracket_2 \gg f$), introducing a systematic bias into the result. This NEV has gone unnoticed because the wrong implementation looks almost identical to the correct one, yet its impact is significant: when training a linear classifier on the Digits dataset [2], this NEV reduces model accuracy by nearly 20 percentage points. In critical applications like medical diagnosis, such an NEV may cause a diagnostic model to consistently misclassify patient conditions.

However, since most MPC implementations rely on probabilistic truncation protocols to handle real-number computation, detecting NEVs faces a fundamental challenge: when we observe a discrepancy between the output of an MPC implementation and the corresponding real-number result, it is difficult to determine whether the discrepancy stems from NEVs or just from random truncation noise, which is introduced by probabilistic truncation protocols. Specifically, MPC implementations typically employ fixed-point arithmetic to support computation on real numbers. Operations such as secure multiplication on fixed-point numbers require truncation to maintain consistent precision, which introduces random truncation noise. The truncation noise is inherent—it is a byproduct of the protocol design, not implementation flaws. However, after a series of computations, the noise accumulates in unpredictable ways. This creates a fundamental ambiguity: when we observe a discrepancy between the output of an MPC implementation and the corresponding real-number result, we cannot determine whether this discrepancy represents (1) truncation noise inherent to the protocol, or (2) a genuine NEV. A naive approach is to compare each MPC output against its real-number result and flag an NEV whenever their difference exceeds a fixed threshold. However, this approach fails to resolve the ambiguity: the magnitude of truncation noise varies with the number of operations, the input values, and the protocol parameters, making any fixed threshold unreliable—a strict threshold flags truncation noise as NEVs, while a loose threshold misses real NEVs.

To address this fundamental challenge, we propose MPCArbiter, a practical framework for detecting NEVs in MPC implementations by establishing a distribution-aware differential oracle. Our key insight is that while truncation noise varies randomly across individual executions, correct MPC implementations of the same MPC protocol with the same input must produce statistically identical output distributions when executed numerous times. This statistical property provides a reliable basis for distinguishing NEVs from truncation noise. Besides, we observe that many open-source MPC frameworks implement the same fundamental arithmetic operations, providing a natural basis for differential testing [21] (a technique that detects vulnerabilities by comparing the behaviors

of different implementations of the same software specification, detailed in Section 2.3). Our proposed oracle operates as follows: (1) Instead of comparing the output of an MPC implementation against the corresponding real-number result (which lacks truncation noise and thus provides no basis for comparison), our proposed oracle performs differential testing between two distinct MPC implementations of the same MPC protocol. (2) Instead of comparing a single output (which will almost certainly differ due to random truncation noise), our proposed oracle executes the implementations numerous times with the same input to construct their empirical output distributions. (3) Our proposed oracle applies the two-sample Kolmogorov-Smirnov test [5] (KS test) to determine whether two distributions are statistically significantly different. If two distributions are statistically significantly different, we conclude that at least one implementation contains NEVs. This approach achieves controllable false positive rates by setting the significance level parameter α in the KS test (detailed in Section 2.4).

Beyond the distribution-aware differential oracle, we complement MPCArbiter with an NEV localization method based on MPC operation dependency graph to reduce debugging effort. Our key insight is that NEVs exhibit error propagation along the data dependency path: once an incorrect operation executes, all dependent operations will propagate this error. Therefore, our localization proceeds in three steps: (1) we construct an MPC operation dependency graph to capture data flow, where nodes represent MPC operations and edges represent data dependencies, and instrument each node in the graph to reveal and trace intermediate values during execution; (2) we apply the KS test on the distribution of intermediate values to identify the first divergence node—the earliest node where implementations produce statistically different distributions; (3) we eliminate nodes unrelated to the first divergence node, substantially reducing the code region to inspect.

We implement MPCArbiter and conduct evaluation on 36 MPC implementations of four mainstream MPC frameworks: CryptTen, TF-Encrypted, MP-SPDZ, and SecretFlow. Our evaluation discovers 10 unique NEVs across diverse secret sharing schemes and operations. Notably, our findings reveal a widespread oversight in the MPC research community regarding probabilistic truncation protocols (detailed in Section 3.2): multiple influential research papers [12, 15, 22, 25, 28] incorrectly employ symmetric truncation in their protocol designs, unaware that this introduces significantly larger systematic bias compared to asymmetric truncation. This design-level error has propagated into multiple MPC frameworks (including SecretFlow [29] and TF-Encrypted [6]). All detected vulnerabilities have been responsibly disclosed to the respective developers. To date, most vulnerabilities have been confirmed and repaired. These results demonstrate not only the prevalence of NEVs in production MPC frameworks but also MPCArbiter’s capability to detect subtle vulnerabilities that span from protocol design to implementation.

We summarize the main contributions of this paper as follows:

- We propose a distribution-aware differential oracle to distinguish NEVs from truncation noise with controllable false positive rates.
- We design an NEV localization method based on MPC operation dependency graph to reduce debugging effort.

- We evaluate MPCArbiter on 36 MPC implementations of four mainstream MPC frameworks, discovering 10 unique NEVs.
- We reveal a widespread oversight in the MPC research community regarding probabilistic truncation protocols: multiple influential research papers [12, 15, 22, 25, 28] incorrectly employ symmetric truncation in their protocol designs, unaware that this introduces significantly larger systematic bias compared to asymmetric truncation. This design-level error has propagated from research papers into multiple MPC frameworks, including SecretFlow [29] and TF-Encrypted [6].

2 Background

2.1 Brief Overview of MPC

MPC [10, 33] is a cryptographic technique that allows a group of mutually distrusting parties to jointly compute a function over their individual inputs, while only revealing the output of the function. Formally, for a set of parties $\{P_1, P_2, \dots, P_n\}$, each holding a private input x_i , an MPC protocol enables them to jointly compute a function f over their inputs $(x_1, x_2, \dots, x_n)$, resulting in outputs $(y_1, y_2, \dots, y_n) = f(x_1, x_2, \dots, x_n)$. The protocol guarantees that no party learns any additional information about the inputs or outputs of other parties, aside from its own output y_i and any information that can be inferred from y_i .

A foundational technique for MPC is secret sharing, which is a cryptographic method to distribute a secret value among multiple parties. There exist many schemes of secret sharing, such as additive secret sharing (ASS) [8], replicated secret sharing (RSS) [22], and Boolean secret sharing (BSS) [8]. For illustration, we use the two-party ASS as a representative example. In two-party ASS, a secret value x is split into two random shares, denoted as $\llbracket x \rrbracket = (\llbracket x \rrbracket_1, \llbracket x \rrbracket_2)$, where P_1 holds $\llbracket x \rrbracket_1$ and P_2 holds $\llbracket x \rrbracket_2$. The shares are generated such that they sum to the secret, i.e., $x = \llbracket x \rrbracket_1 + \llbracket x \rrbracket_2 \pmod{p}$. Here, p is the size of the computation domain. In two-party ASS, any individual share $\llbracket x \rrbracket_i$ reveals no information about x on its own, and the secret can only be reconstructed when the two parties combine their shares.

MPC protocols are typically constructed by performing computations on secret-shared values. For illustration, we take the secure addition protocol in two-party ASS as an example. Assume P_1 and P_2 hold the shares of two secrets x and y . The secure addition protocol in two-party ASS is simple and non-interactive: parties compute a share of the sum $\llbracket x + y \rrbracket$ by locally adding their respective shares $(\llbracket x \rrbracket_i + \llbracket y \rrbracket_i)$.

Throughout this paper, we use the term "MPC operation" to refer to basic MPC protocols. For instance, we use "secure addition operation" to refer to the secure addition protocol that adds two secret-shared values, and "secure comparison operation" to refer to the secure comparison protocol that compares secret-shared values, and so forth.

By combining MPC operations, multiple mutually distrusting parties can perform a wide array of joint data analysis tasks, such as training various machine learning models [19, 20, 23]—while keeping their data private. This capability helps bridge the gap between data utility and data privacy, making MPC protocols a cornerstone for joint data analysis on critical and sensitive information.

2.2 Fixed-Point Arithmetic in MPC

A primary challenge in applying MPC protocols to real-world tasks, such as machine learning, is that MPC is fundamentally designed for computations over discrete algebraic structures, typically integer rings ($\mathbb{Z}_{2^k}$) or finite fields ($\mathbb{Z}_p$). However, most data analysis and machine learning models rely on real numbers.

To bridge the gap, MPC frameworks usually adopt fixed-point arithmetic. This technique simulates real numbers by encoding them as large integers. A real number $\tilde{x}$ is encoded into an integer x by multiplying it by a scaling factor, typically a power of two, 2^f , and then rounding to the nearest integer: $x = \text{round}(\tilde{x} \cdot 2^f)$. The value f is known as the fractional bits. For example, if we choose $f = 10$, the real number "3.5" would be encoded as the integer $3.5 \cdot 2^{10} = 3584$.

This encoding introduces a major complication: to maintain consistent fractional bits, operations like secure multiplication require a subsequent truncation protocol, which inherently introduces truncation noise. Specifically, when two fixed-point shared integers $\llbracket x \rrbracket$ and $\llbracket y \rrbracket$, each with f fractional bits, are securely multiplied to produce $\llbracket z \rrbracket = \llbracket x \rrbracket \cdot \llbracket y \rrbracket$, the resulting shared integer $\llbracket z \rrbracket$ has $2f$ fractional bits and must be truncated back to f bits before subsequent operations. To maintain privacy, this truncation is performed directly on the secret shares rather than on the reconstructed value. To illustrate, consider a two-party ASS scheme where the product z is shared as $\llbracket z \rrbracket = (\llbracket z \rrbracket_1, \llbracket z \rrbracket_2)$, a common truncation protocol involves each party locally truncating its share: P_1 computes $\llbracket z' \rrbracket_1 = \llbracket z \rrbracket_1 \gg f$, while P_2 computes $\llbracket z' \rrbracket_2 = -((- \llbracket z \rrbracket_2) \gg f)$, where $x \gg f$ denotes arithmetic right shift of x by f bits.

Since each party right-shifts its own share independently, the carry generated by adding the lower f bits of the two shares is lost. As a result, the truncated value may be rounded either up or down, depending on the random shares. For example, let $f = 2$ and the value to be truncated be $z = 11$ (with $2f = 4$ fractional bits). The exact quotient $11/2^2 = 2.75$ is not an integer, so the result must be rounded to either 2 or 3, and which one occurs depends on the shares:

- *Case 1:* $z = 11$ is shared as $\llbracket z \rrbracket_1 = 3$ and $\llbracket z \rrbracket_2 = 8$.
 - P_1 Truncates: $\llbracket z' \rrbracket_1 = \llbracket z \rrbracket_1 \gg f = 3 \gg 2 = 0$.
 - P_2 Truncates: $\llbracket z' \rrbracket_2 = -((- \llbracket z \rrbracket_2) \gg f) = -((-8) \gg 2) = -(-2) = 2$.
 - Result: The new shares sum to $z' = 0 + 2 = 2$, i.e., the exact quotient is rounded down.
- *Case 2:* $z = 11$ is shared as $\llbracket z \rrbracket_1 = 8$ and $\llbracket z \rrbracket_2 = 3$.
 - P_1 Truncates: $\llbracket z' \rrbracket_1 = \llbracket z \rrbracket_1 \gg f = 8 \gg 2 = 2$.
 - P_2 Truncates: $\llbracket z' \rrbracket_2 = -((- \llbracket z \rrbracket_2) \gg f) = -((-3) \gg 2) = -(-1) = 1$.
 - Result: The new shares sum to $z' = 2 + 1 = 3$, i.e., the exact quotient is rounded up.

As shown, depending on the random shares, the protocol outputs either 2 or 3, differing by $1 (\frac{1}{2^f})$ in real number representation). This deviation, which we refer to as truncation noise, is inherent to the protocol design rather than an implementation flaw.

However, the seemingly minor noise can be significantly amplified through subsequent operations. For example, an initial truncation noise of $\frac{1}{2^{10}}$, when multiplied by another value of 2^{20} , will be

amplified to 2^{10} . In complex computations involving many operations, truncation noise continuously accumulates in unpredictable ways, making it extremely difficult to distinguish NEVs from accumulated noise—this is the fundamental challenge in detecting NEVs in MPC implementations.

2.3 Differential Testing

Differential testing is a testing technique that detects bugs by comparing the behaviors of multiple software implementations of the same specification [21]. The underlying assumption, often called the N-version programming hypothesis, is that different software implementations are unlikely to contain the same bug in the same way. Differential testing works as follows: the same input x is fed to two or more software implementations, and their behaviors are compared—a discrepancy in behavior flags a potential bug in at least one implementation.

Differential testing is particularly valuable when a test ground truth is hard to establish. For example, when testing compiler implementations, establishing ground truth requires manually verifying whether the generated assembly code correctly implements the source program semantics—a tedious and error-prone process for complex programs. Instead, developers feed the same source code to multiple compilers (e.g., GCC, Clang, MSVC) and compare their generated binaries or runtime behaviors—any discrepancy indicates a bug in at least one compiler, without needing to establish what the “correct” output should be.

The existence of numerous open-source MPC frameworks provides a natural opportunity for differential testing. However, directly applying differential testing to detect NEVs faces a fundamental challenge: how to reliably identify NEVs in the presence of truncation noise. Specifically, due to random truncation noise, two correct MPC implementations of the same protocol will produce different outputs on every execution. Besides, the truncation noise magnitude varies with the number of operations, input values, and protocol parameters, making any threshold-based differential test approaches unreliable—setting the threshold too strict flags truncation noise as NEVs, while setting the threshold too loose misses NEVs.

2.4 Two-Sample Kolmogorov-Smirnov Test

The KS test is a non-parametric statistical test that determines whether two sample sets are drawn from the same underlying distribution. Unlike parametric tests that assume specific distribution shapes (e.g., normal distributions), the KS test is distribution-free and makes no assumptions about the data’s form, making it widely applicable across diverse scenarios.

The KS test operates by comparing the empirical cumulative distribution functions (ECDFs) of two sample sets. An ECDF represents the proportion of data points in a sample set that fall at or below each value. The KS test computes the maximum vertical distance between the two ECDFs—a large distance suggests that the two sample sets likely originate from different distributions, while a small distance indicates they may come from the same distribution.

The KS test then produces a p -value based on the observed maximum distance and the sample sizes to assess statistical significance. The p -value represents the probability of observing such a large

distance if both sample sets truly came from the same distribution. A small p -value provides strong evidence that the distributions differ, while a large p -value suggests the observed difference could easily arise by random chance.

Decision-making with the KS test involves comparing the p -value against a predetermined significance level α (commonly set to 0.01 or 0.05). If $p_{value} < \alpha$, we conclude that both sample sets are statistically different. Otherwise, we treat the two sample sets as statistically indistinguishable. The significance level α serves as a controllable threshold: smaller values (e.g., 0.01) provide higher confidence in detected differences but may miss subtle ones, while larger values (e.g., 0.05) offer greater sensitivity at the cost of occasionally flagging chance differences.

3 Numeric Error Vulnerability Examples

To illustrate the concept of NEV, we first present a representative vulnerability discovered by MPCArbiter in SecretFlow’s ASS truncation protocol implementation. Then, we reveal that this NEV is not unique to SecretFlow but exists widely in truncation protocol designs.

3.1 NEV in SecretFlow’s ASS Truncation

Root Cause. As specified in Section 2.2, the correct truncation protocol for two-party ASS requires parties to perform asymmetric truncation on their shares: P_1 computes $\llbracket z' \rrbracket_1 = \llbracket z \rrbracket_1 \gg f$ (equal to $\lfloor \llbracket z \rrbracket_1 / 2^f \rrbracket$), and P_2 computes $\llbracket z' \rrbracket_2 = -((- \llbracket z \rrbracket_2) \gg f)$ (equal to $\lceil \llbracket z \rrbracket_2 / 2^f \rceil$). However, MPCArbiter discovers that SecretFlow’s ASS implementation erroneously applies symmetric truncation: P_1 computes $\llbracket z' \rrbracket_1 = \llbracket z \rrbracket_1 \gg f$ and P_2 computes $\llbracket z' \rrbracket_2 = \llbracket z \rrbracket_2 \gg f$.

The likely origin of this NEV may stem from implementation simplicity—symmetric truncation is more intuitive and easier to implement. Developers may have assumed that since both floor and ceiling operations introduce errors, the choice between them would not significantly matter.

However, the assumption is fundamentally incorrect: asymmetric truncation and symmetric truncation have drastically different statistical properties in terms of their expected absolute deviation from the exact real-number result.

Illustrative Example. To understand why asymmetric truncation and symmetric truncation have different absolute deviation from the exact real-number result, consider a concrete example. Suppose $z = 11$ with $2f = 4$ fractional bits represents the real number $T_{real} = z/2^{2f} = 11/16 = 0.6875$. The secret $z = 11$ can be randomly shared in different ways:

Case 1: $z = 11$ is shared as $\llbracket z \rrbracket_1 = 3$, $\llbracket z \rrbracket_2 = 8$

- **Symmetric truncation (SecretFlow):**

$$T_B = (\llbracket z \rrbracket_1 \gg 2) + (\llbracket z \rrbracket_2 \gg 2) = (3 \gg 2) + (8 \gg 2) = 0 + 2 = 2$$

Converting to real number: $T_B/2^f = 2/4 = 0.5$

Absolute deviation: $|0.5 - 0.6875| = 0.1875$

- **Asymmetric truncation (Correct):**

$$\begin{aligned} T_C &= (\llbracket z \rrbracket_1 \gg 2) + (-((- \llbracket z \rrbracket_2) \gg 2)) \\ &= (3 \gg 2) + (-((-8) \gg 2)) = 0 + (-(-2)) = 2 \end{aligned}$$

Converting to real number: $T_C/2^f = 2/4 = 0.5$

Absolute deviation: $|0.5 - 0.6875| = 0.1875$

In this case, both methods produce the same result with absolute deviation 0.1875.

Case 2: $z = 11$ is shared as $\llbracket z \rrbracket_1 = 8, \llbracket z \rrbracket_2 = 3$

- **Symmetric truncation (SecretFlow):**

$$T_B = (\llbracket z \rrbracket_1 \gg 2) + (\llbracket z \rrbracket_2 \gg 2) = (8 \gg 2) + (3 \gg 2) = 2 + 0 = 2$$

Converting to real number: $T_B/2^f = 2/4 = 0.5$

Absolute deviation: $|0.5 - 0.6875| = 0.1875$

- **Asymmetric truncation (Correct):**

$$\begin{aligned} T_C &= (\llbracket z \rrbracket_1 \gg 2) + (-((-\llbracket z \rrbracket_2) \gg 2)) \\ &= (8 \gg 2) + (-((-3) \gg 2)) = 2 + (-(-1)) = 3 \end{aligned}$$

Converting to real number: $T_C/2^f = 3/4 = 0.75$

Absolute deviation: $|0.75 - 0.6875| = 0.0625$

In this case, symmetric truncation yields an absolute deviation of 0.1875, whereas asymmetric truncation yields 0.0625—smaller than 0.1875.

Statistical Analysis. Let $T_{real} = z/2^{2f}$ be the exact real-number result. Statistical analysis (detailed derivation in Appendix D) shows:

For SecretFlow’s symmetric (erroneous) implementation:

$$\mathbb{E}[|T_B/2^f - T_{real}|] = \frac{2^f - 1}{2^{2f}} \approx \frac{1}{2^f}$$

For the asymmetric (correct) specification:

$$\mathbb{E}[|T_C/2^f - T_{real}|] = \frac{2^{2f} - 1}{3 \cdot 2^{3f}} \approx \frac{1}{3 \cdot 2^f}$$

The symmetric truncation has approximately $3\times$ larger expected absolute deviation per truncation operation compared to asymmetric truncation. For example, when $f = 10$, symmetric truncation introduces an expected absolute deviation of ≈ 0.001 per operation, while asymmetric truncation introduces only ≈ 0.0003 .

Impact. As shown in Figure 1, we empirically demonstrate the severe impact of this biased truncation NEV by comparing the erroneous SecretFlow implementation against a correct TF-Encrypted implementation:

- **Arithmetic Impact.** Figure 1a highlights the deviation in a purely arithmetic task. We conduct 1,000 independent trials of 30 sequential secure multiplication operations with the same input. The erroneous SecretFlow implementation’s output distribution is visibly shifted, with its mean (0.3468) deviating significantly from the float baseline (0.2800), while TF-Encrypted’s mean (0.2705) remains statistically aligned with the baseline.
- **Machine Learning Impact.** Figure 1b shows the test accuracy of a linear classifier trained on the Digits dataset[2]: TF-Encrypted converges quickly to baseline accuracy ($\approx 85\%$), while SecretFlow plateaus at 66% due to accumulating error. Figure 1c shows that SecretFlow requires high fractional bits ($f = 14$) to converge, while TF-Encrypted achieves high accuracy even at low fractional bits ($f = 7$).

3.2 Widespread NEV in Truncation Design

Even more concerning, the NEV we discover in SecretFlow’s probabilistic truncation implementation is not unique to this framework. Our investigation reveals that symmetric truncation—the

root cause of this vulnerability—is a widespread design-level error in multiple influential research papers [12, 15, 22, 25, 28] of MPC.

To illustrate this design-level NEV, we use the truncation protocol in the ABY3 paper [22] as an example. In ABY3, a secret z is shared as three shares $\llbracket z \rrbracket_1, \llbracket z \rrbracket_2, \llbracket z \rrbracket_3$ where $z = \llbracket z \rrbracket_1 + \llbracket z \rrbracket_2 + \llbracket z \rrbracket_3 \pmod{p}$. The protocol specifies truncating z by computing $(\llbracket z \rrbracket_1 + \llbracket z \rrbracket_2) \gg f$ and $\llbracket z \rrbracket_3 \gg f$. This truncation introduces a systematic bias analogous to the erroneous two-party ASS truncation approach, where both parties apply symmetric truncation $(\llbracket z \rrbracket_1 \gg f, \llbracket z \rrbracket_2 \gg f)$, introducing the $3\times$ larger error compared to asymmetric truncation.

The design-level NEV in ABY3 has been faithfully implemented in the RSS scheme of SecretFlow [29] and TF-Encrypted [6].

4 Design of MPCArbiter

4.1 Overview

Workflow. Figure 2 shows the workflow of MPCArbiter, which takes as input two MPC implementations under test. (1) MPCArbiter first generates a test input x . This input is then passed to the distribution-aware differential oracle. The oracle executes both implementations numerous times with x to construct output distributions, then applies the KS test to determine if the distributions are significantly different. If the KS test indicates a statistically significant difference (i.e., an NEV is detected), MPCArbiter records the vulnerability-triggering input as x_{err} . (2) MPCArbiter further invokes the NEV localization algorithm to narrow down the code regions responsible for the NEV through graph construction and instrumentation, divergence node search, and graph pruning, producing two suspicious code regions: $C_{suspect-A}$ for implementation A and $C_{suspect-B}$ for implementation B. Finally, MPCArbiter returns the tuple $(x_{err}, C_{suspect-A}, C_{suspect-B})$ as the NEV report.

Applicable Protocols. Since MPCArbiter’s oracle compares the two implementations’ output distributions and is agnostic to the protocol’s internal logic, MPCArbiter applies to any MPC protocol that has two independent implementations, including arithmetic operations, comparison operations, and other common PPML building blocks like division and lookup tables.

4.2 Distribution-Aware Differential Oracle

Challenge. The fundamental challenge for detecting NEVs lies in designing an oracle that can accurately distinguish genuine NEVs from truncation noise. As discussed in Section 2.2, MPC implementations extensively use fixed-point arithmetic to support real number computations. Operations like secure multiplication require truncation, which introduces random truncation noise. This creates a critical ambiguity: when we observe a discrepancy between two MPC implementations (or between an MPC implementation and its plaintext counterpart), we cannot immediately determine whether this discrepancy represents (1) truncation noise inherent to the protocol, or (2) a genuine NEV caused by an implementation flaw. A naive approach that compares MPC outputs against real-number results using fixed thresholds fails to address this ambiguity: truncation noise magnitude varies depending on the number of operations, input values, and protocol parameters, making any fixed threshold unreliable—setting it too strict will flag truncation noise as NEVs, while setting it too loose will miss NEVs.

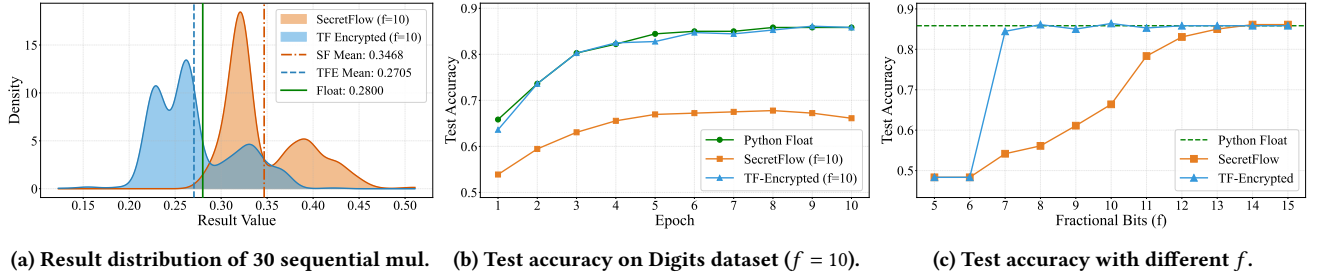


Figure 1: Comparison of numerical behaviors of truncation in ASS scheme between SecretFlow (symmetric truncation) and TF-Encrypted (asymmetric truncation). (a) Output distributions of 1000 trials of 30 sequential secure multiplication operations with the same input, showing that SecretFlow’s mean (0.3468) deviates significantly from the plaintext float (0.2800), while TF-Encrypted’s mean (0.2705) remains much closer. (b) Test accuracy on the Digits dataset over 10 epochs with $f = 10$. SecretFlow shows slower convergence and lower accuracy. (c) Test accuracy after 10 epochs with varying fractional bits, showing that SecretFlow requires more fractional bits to match plaintext float accuracy than TF-Encrypted.

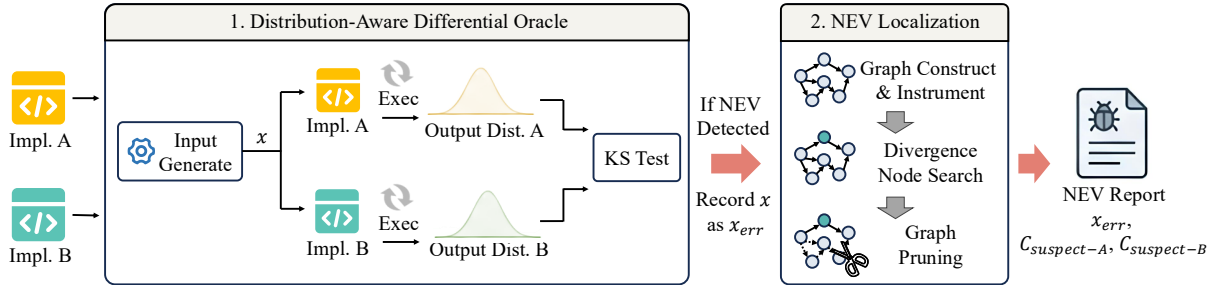


Figure 2: Workflow of MPCArbiter. Distribution-Aware Differential Oracle: For each test input x , both implementations (Impl. A and Impl. B) are executed numerous times (Exec) to construct output distributions (Output Dist. A and Dist. B), which are compared using the KS test. **NEV Localization:** When an NEV is detected with triggering input x_{err} , this component constructs and instruments MPC operation dependency graph for the implementations, performs divergence node search and graph pruning on the graph to narrow down the suspicious code regions $C_{suspect-A}$ (for Impl. A) and $C_{suspect-B}$ (for Impl. B). The final output is the NEV report ($x_{err}, C_{suspect-A}, C_{suspect-B}$).

Our Approach. To address this challenge, we establish a distribution-aware differential oracle based on a key insight: while truncation noise varies randomly across individual executions, correct implementations of the same MPC protocol with the same input must produce statistically identical output distributions. This property holds because the truncation noise distribution is uniquely determined by the input value and the protocol specification. Specifically, truncation protocols operate on uniformly random secret shares, and the outcome of truncation (whether rounding up or down occurs) depends probabilistically on how the fractional bits are distributed across shares. Since the secret shares are uniformly random and the input is fixed, this results in a fixed truncation noise distribution, which in turn determines a fixed output distribution for the MPC computation. Consequently, any correct implementation of the same protocol with the same input will produce outputs following an identical distribution, even though individual outputs vary randomly due to the randomness of truncation noise. In contrast, an implementation containing an NEV, e.g., using incorrect rounding logic, will produce a different output distribution. As demonstrated in Section 3.1, the symmetric truncation NEV

introduces a $3\times$ larger expected absolute deviation, fundamentally altering the distribution’s statistical properties.

Based on this insight, our proposed oracle operates as follows: instead of comparing single outputs, we execute each implementation numerous times with the same input to empirically construct their output distributions, and then determine whether these distributions are significantly different. If the distributions are statistically indistinguishable, we conclude that the observed discrepancies are just truncation noise. If the distributions are significantly different, we flag this as evidence of an NEV in at least one implementation.

Note that our proposed distribution-aware differential oracle also applies to detect NEVs in two MPC implementations without truncation (e.g., implementations of a secure sorting protocol). In such cases, if an NEV exists in one implementation, its output values will differ directly from those of the other implementation, making their distributions differ statistically.

Algorithm Description. As shown in Algorithm 1, our proposed oracle takes as input the valid input range (V_{min}, V_{max}), two MPC implementations I_A and I_B , and two parameters $N_{samples}$ (the number of runs to construct each distribution) and α (the significance

level for the KS test). The algorithm outputs a boolean indicating whether an NEV is detected.

The oracle first generates a test input x from the valid range (line 1). The oracle then initializes two empty distributions D_A and D_B (lines 2-3). It executes both implementations I_A and I_B repeatedly with the same fixed input x for $N_{samples}$ times, collecting the outputs into distribution D_A and D_B (lines 4-9). After constructing both distributions, the oracle applies the KS test to measure the difference between the two distributions (line 10). The KS test returns p_{value} , the probability that the two distributions are identical. The oracle then checks whether $p_{value} < \alpha$ (line 11). If this condition holds, the oracle returns True to flag an NEV (line 12). Otherwise, the oracle returns False (line 14).

Algorithm 1 Distribution-Aware Differential Oracle

Input: The valid input range (V_{min}, V_{max}); and the two implementations (I_A, I_B).

Output: A boolean indicating if an NEV is detected (is_nev).

Parameters: The number of runs for each implementation ($N_{samples}$); and the significance level (α).

```

1:  $x \leftarrow \text{InputGenerate}(V_{min}, V_{max})$            ▶ Generate test input
2:  $D_A \leftarrow \emptyset$                              ▶ Initialize distribution for  $I_A$ 
3:  $D_B \leftarrow \emptyset$                              ▶ Initialize distribution for  $I_B$ 
4: for  $i = 1$  to  $N_{samples}$  do
5:    $o_A \leftarrow I_A(x)$ 
6:    $D_A \leftarrow D_A \cup \{o_A\}$ 
7:    $o_B \leftarrow I_B(x)$ 
8:    $D_B \leftarrow D_B \cup \{o_B\}$ 
9: end for
10:  $p_{value} \leftarrow \text{KStest}(D_A, D_B)$ 
11: if  $p_{value} < \alpha$  then
12:   return True                                     ▶ NEV detected
13: else
14:   return False                                   ▶ No NEV detected
15: end if

```

Input Generation Strategy. The oracle requires a test input, which MPCArbiter generates with an evolutionary strategy based on NSGA-II [7]. This evolutionary strategy jointly optimizes two objectives: the output discrepancy $|\bar{y}_A(x) - \bar{y}_B(x)|$, where $\bar{y}_A(x)$ and $\bar{y}_B(x)$ denote the average outputs of the two implementations over multiple runs, and the input diversity. This design is guided by two intuitions: (1) Since the KS test flags an NEV only when the two output distributions differ noticeably, an input that induces a larger output discrepancy is more likely to expose a detectable NEV. (2) Since some NEVs are triggered only in specific regions of the input space, maintaining input diversity helps the search cover these regions.

4.3 NEV Localization

Once MPCArbiter detects an NEV, manually debugging complex MPC implementations with hundreds of lines of cryptographic operations is time-consuming. To reduce the manual debugging effort, we design an NEV localization method to narrow down the suspicious code region responsible for the NEV.

Our Approach. Our key insight is that NEVs exhibit error propagation along the data dependency path: once an MPC operation produces an error output, all operations that depend on its output will propagate this error. By constructing an MPC operation dependency graph to capture data flow relationships and identifying the first divergence node where implementations produce statistically different outputs, we can significantly narrow down the suspicious code region.

Our approach consists of three main steps: (1) *Dependency Graph Construction and Instrumentation*—we construct a dependency graph where nodes represent MPC operations and edges represent data dependencies, and instrument each operation node to reveal intermediate values; (2) *Divergence Node Search*—we locate the first node where outputs diverge using KS testing; (3) *Graph Pruning*—we eliminate nodes unrelated to the divergence node to narrow the suspect code region.

Illustrative Example. Figure 3 shows the complete localization process through a concrete example. Since both implementations execute the same MPC protocol, they should share the same MPC operation dependency graph. Due to space limitations, we show the code for only one implementation, though the algorithm processes both implementations and produces separate suspicious code regions for each.

Figure 3a shows the original implementation with 10 MPC operations. Figure 3b displays the dependency graph with 9 nodes—Node N7 is excluded because its result does not contribute to the final output. Figure 3c shows the pruned graph after localization identifies Node N6 ($z_3 = z_2 + x$) as the first divergence node by comparing the intermediate value distributions between the two implementations. The algorithm eliminates two types of nodes: (1) nodes after the divergence node (N9-N10), which just propagate the error without causing it; (2) nodes on unrelated branches (N4, N8), which do not contribute to the divergence at N6. Figure 3d shows the final suspicious code region, containing only the operations on paths leading to Node N6 (N1-N3, N5-N6), with pruned and unused operations shown as comments for reference.

Algorithm 2 NEV Localization

Input: Vulnerability-triggering input (x_{err}); implementations (I_A, I_B).

Output: Suspicious code regions ($C_{suspect-A}, C_{suspect-B}$).

Parameters: Number of runs ($N_{samples}$); significance level (α).

```

1:  $G \leftarrow$  Construct dependency graph for the MPC protocol
2:  $I'_A, I'_B \leftarrow$  Instrument the operation nodes in  $G$  into  $I_A, I_B$ 
3:  $All\_Results_A, All\_Results_B \leftarrow \text{Matrix}(N_{samples}, M)$ 
4: for  $i = 0$  to  $N_{samples} - 1$  do
5:    $\vec{v}_A \leftarrow I'_A(x_{err}); \vec{v}_B \leftarrow I'_B(x_{err})$ 
6:    $All\_Results_A[i] \leftarrow \vec{v}_A; All\_Results_B[i] \leftarrow \vec{v}_B$ 
7: end for
8:  $n_{diverge} \leftarrow$  Binary search on topological order of  $G$  to find first divergence node using KS test
9:  $Ancestors \leftarrow$  Backward reachability from  $n_{diverge}$  in  $G$ 
10:  $C_{suspect-A}, C_{suspect-B} \leftarrow$  Code snippets corresponding to  $Ancestors$  in  $I_A$  and  $I_B$ , respectively.
11: return ( $C_{suspect-A}, C_{suspect-B}$ )

```

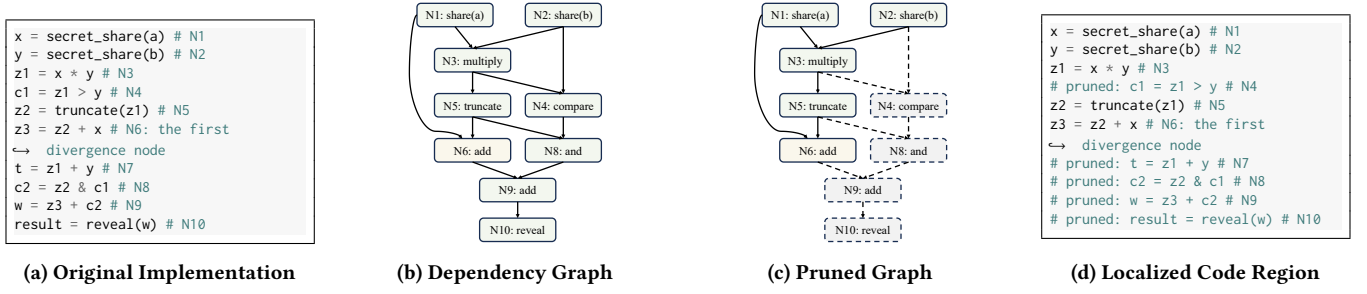


Figure 3: NEV localization example (only one implementation shown for space). (a) Original implementation with 10 MPC operations. (b) Dependency graph with 9 nodes (N7 excluded as unrelated to output). (c) Pruned graph after identifying N6 as the first divergence node through the KS test. Gray dashed nodes indicate pruned operations. (d) Final suspicious code region containing only operations (N1-N3, N5-N6) on paths leading to the divergence node.

Algorithm Description. As shown in Algorithm 2, our proposed NEV localization method takes a vulnerability-triggering input x_{err} and two implementations I_A, I_B as input, and outputs two suspicious code regions $C_{suspect-A}$ (for I_A) and $C_{suspect-B}$ (for I_B).

Step 1: Dependency Graph Construction and Instrumentation. We construct an MPC operation dependency graph G that represents the computational structure common to both implementations (line 1). Since both implementations execute the same MPC protocol, i.e., they perform the same sequence of MPC operations (e.g., secure multiplication, truncation, comparison) in the same order, they should share the same MPC operation dependency graph. We construct this dependency graph by analyzing the data flow: starting from the final output, we trace through all intermediate operations in reverse order to identify producer-consumer relationships. Each MPC operation becomes a node, and we create a directed edge $op_j \rightarrow op_i$ when operation op_j produces a value consumed by operation op_i . This process identifies M relevant operation nodes that produce intermediate values. We then instrument both implementations to reveal intermediate values at each of these M operation nodes (line 2).

Step 2: Divergence Node Search. To search the first divergence node, we first collect intermediate values by running the instrumented implementations $N_{samples}$ times with input x_{err} (lines 3-7). Each column in the resulting matrices represents the distribution of one operation node across multiple runs. Using these collected intermediate values, we search for the first divergence node in the dependency graph (line 8). Since the graphs may have multiple branches, we process nodes in topological order and perform binary search on this ordering to efficiently locate the first divergence node. At each step, we test the midpoint node in the topological order using the KS test: if $p_{value} < \alpha$ (indicating the two distributions are statistically significantly different at this node), divergence occurred at or before this node, so we search in the earlier portion of the topological order; otherwise, we search in the later portion. This identifies the first divergence node $n_{diverge}$.

Step 3: Graph Pruning. We perform backward reachability analysis from the divergence node to identify its ancestors, i.e., all nodes on paths leading to it (line 9). This eliminates two types of nodes: (1) nodes after the first divergence node in topological order, as they just propagate the error rather than causing it; (2)

nodes on branches unrelated to the divergence node, as they do not contribute to the vulnerability. The remaining nodes in each implementation form $C_{suspect-A}$ and $C_{suspect-B}$ respectively (line 10), reducing the code region requiring manual debugging.

Localize Multiple NEVs. When multiple NEVs exist in an implementation, we can localize them through an iterative fix-and-retest workflow: localize and fix the first NEV, and then rerun MPCArbiter on the implementations to localize the next one, repeating until no more NEVs are found. This iterative approach is necessary because during localization, the later NEVs are masked by earlier ones.

5 Evaluation

In this section, we evaluate MPCArbiter to answer the following research questions:

- **RQ1:** How effective is MPCArbiter in detecting NEVs in real-world MPC frameworks?
- **RQ2:** What are the root causes and impact of the detected NEVs?
- **RQ3:** What is the time efficiency of MPCArbiter in detecting NEVs?

5.1 Implementation and Experiment Settings

Testing Frameworks and Schemes. We evaluate MPCArbiter in four widely-used open-source MPC frameworks.

- **CrypTen** [14] (1.6k+ GitHub stars): Developed by Facebook AI Research, CrypTen is a PyTorch-based MPC framework designed for machine learning applications. We test its ASS and RSS schemes, which are realized by the two-party beaver backend and the three-party replicated backend, respectively.
- **TF-Encrypted** [6] (1.2k+ GitHub stars): Maintained by Cape Privacy, OpenMined, and Alibaba, TF-Encrypted is a privacy-preserving machine learning framework based on TensorFlow. We test its ASS and RSS schemes, which are realized by the securenn backend and the aby3 backend, respectively.
- **MP-SPDZ** [13] (1.1k+ GitHub stars): Developed by CSIRO's Data61, MP-SPDZ is a versatile MPC framework supporting over 40 different secret sharing schemes. We test its ASS, RSS, and BSS schemes, which are realized by the semi2k-party.x,

replicated-ring-party.x, and replicated-bin-party.x virtual machines, respectively.

- **SecretFlow** [29] (2.6k+ GitHub stars): Developed by Ant Group, SecretFlow is an industrial-grade privacy-preserving computing framework designed for production deployments. We test its ASS and RSS schemes, which are realized by the SEMI2K and ABY3 backends, respectively.

Throughout this paper, we use notation like `CrypTen-ASS` to refer to the ASS scheme in `CrypTen`, and similarly for other frameworks and schemes. All experiments are conducted under the semi-honest model, with an honest majority for the three-party schemes.

Tested Protocols. We use the following four protocols to test the four frameworks. These protocols cover the fundamental MPC operations commonly used in practice: arithmetic operations (`EleMult`, `MatMult`) and comparison-based operations (`Cmp`, `Sort`). The pseudocode of these protocols is shown in Appendix C.

- `EleMult`: Securely computes the product of 10 secret-shared values.
- `MatMult`: Performs 10 iterations of secure matrix multiplication operations on 2×2 matrices, followed by summation of all elements.
- `Cmp`: Computes multiple secure comparison operations on two secret-shared values.
- `Sort`: Securely sorts an array of 5 secret-shared values in ascending order.

Experimental Environment. All experiments are conducted on a server equipped with Intel Xeon Gold 6230R @ 2.10GHz (20 cores), 128GB DDR4 RAM, running Ubuntu 20.04 LTS (Linux kernel 5.15.0) with Python 3.10.12.

Configuration. For each MPC implementation, we use 3 different inputs generated by our proposed evolutionary generation strategy (detailed in Section 4.2) to test it. The NSGA-II optimization in the evolutionary generation strategy runs with a population size of 10 and 3 generations. For differential testing, we use the MPC implementations in MP-SPDZ-RSS as the reference and compare the implementations in other frameworks and schemes against them. For statistical testing, we perform 500 runs per input ($N_{samples} = 500$) with significance level $\alpha = 0.001$.

5.2 RQ1: Effectiveness of MPCArbiter

As shown in Table 1, we present the NEV detection results across four mainstream MPC frameworks. Our evaluation demonstrates MPCArbiter’s effectiveness through two key dimensions: high detection capability and empirically zero false positives with a reasonable significance level.

Strong Detection Capability. MPCArbiter successfully detects 10 unique NEVs across all four tested frameworks, demonstrating its ability to identify NEVs ranging from subtle biases to large-magnitude errors. Critically, MPCArbiter’s statistical approach enables detection of NEVs even when error magnitudes are small. For instance, SecretFlow-RSS in the EleMult protocol shows $|\Delta| = 9.5e-3$ ($|\Delta|$: mean absolute difference between two output distributions), yet MPCArbiter correctly identifies this as an NEV with $p = 7.4e-300$, distinguishing it from truncation noise. This detection capability stems from MPCArbiter’s core methodology: rather than relying on absolute error thresholds that would dismiss such

small errors as truncation noise, MPCArbiter analyzes the statistical distribution of outputs across hundreds of executions to detect errors that persist regardless of magnitude. At the other extreme, MPCArbiter also detects vulnerabilities with substantial error magnitudes. For example, MP-SPDZ-BSS exhibits large-magnitude errors across all protocols.

Empirically Zero False Positives with a Reasonable Significance Level. With significance level $\alpha = 0.001$, which provides 99.9% confidence, MPCArbiter achieves empirically zero false positives across all 10 detected vulnerabilities. All of these vulnerabilities have been confirmed as genuine implementation errors by developers or our manual verification. The results demonstrate clear separation between NEVs and benign truncation noise in p -values. Implementations with NEVs exhibit extremely low p -values: 11 out of 15 implementations with NEVs show $p = 7.4e-300$ (the smallest p -value reported by our KS test implementation at $N_{samples} = 500$), while others range from $1.0e-44$ to $5.1e-260$, providing overwhelming statistical evidence of systematic vulnerabilities. In stark contrast, implementations without NEVs consistently show high p -values from 0.03 to 1.00, well above the significance threshold. The significant gap in p -values allows MPCArbiter to accurately distinguish NEVs from random truncation noise.

Besides, the detected NEVs span all four frameworks: `CrypTen` (2 NEVs), MP-SPDZ (4 NEVs), TF-Encrypted (2 NEVs), and SecretFlow (2 NEVs). This widespread prevalence across diverse implementations—ranging from research prototypes (MP-SPDZ) to industrial production systems (SecretFlow)—underscores that NEVs are a systemic challenge in the MPC ecosystem and highlights the critical need for systematic detection tools like MPCArbiter.

5.3 RQ2: Root Causes and Impact of NEVs

To understand the root causes and impact of the detected NEVs, we present three representative case studies.¹

5.3.1 Case Study 1: Negative Value Decoding Error (CrypTen-ASS). `CrypTen`’s ASS scheme contains a vulnerability in the fixed-point decoding function that systematically corrupts negative values. For example, decoding -3.5 (encoded as -3584) produces -3.503 instead of the correct -3.5 . This vulnerability remained undetected for years despite `CrypTen` being widely used, precisely because the small errors are extremely difficult to distinguish from truncation noise.

Root Cause. As shown in Figure 4, the `decode()` function in `CrypTen/encoder.py` is responsible for converting fixed-point encoded integers back to floating-point numbers by dividing by the scale factor (2^f). However, the implementation contains a subtle but critical error: it incorrectly decodes negative values by dividing by $(scale - 1)$ instead of $scale$.

The likely origin of this error may stem from an attempt to handle negative number rounding correctly. The developer appears to have tried to apply a "correction" for negative numbers, possibly to compensate for perceived rounding issues with floor division. However, this correction is fundamentally misguided—it changes the divisor itself rather than adjusting the result after division, leading to systematic proportional errors in all negative value decoding.

¹Detailed information on all detected NEVs is available at <https://github.com/FudanMPL/Vulnerabilities-in-MPC-Framework>.

Table 1: NEV Detection Results. Uni Vul: Number of unique NEVs (a single NEV may manifest across multiple protocols/schemes). Res: Detection result; X: discrepancy detected; -: No discrepancy; /: Not implemented or cannot run due to other bugs. $|\Delta|$: Mean absolute difference between two output distributions. Note that $|\Delta|$ varies with the triggering input, and we report the largest observed $|\Delta|$. Besides, MP-SPDZ-RSS is compared against itself using two independent sets of executions; truncation noise makes the two empirical distributions differ slightly in EleMult and MatMult, so its p -value and $|\Delta|$ are not exactly 1.00 and 0.00.

Framework	Uni Vul	Scheme	EleMult			MatMult			Cmp			Sort		
			Res	p -value	$ \Delta $	Res	p -value	$ \Delta $	Res	p -value	$ \Delta $	Res	p -value	$ \Delta $
CrypTen	2	ASS	-	0.67	3.5e-4	X	2.2e-253	12.34	X	7.4e-300	1.00	X	7.4e-300	1.00
		RSS	-	0.51	1.3e-4	/	/	/	-	1.00	0.00	X	7.4e-300	1.00
MP-SPDZ	4	ASS	-	0.67	7.6e-5	-	0.07	0.26	-	1.00	0.00	-	1.00	0.00
		RSS	-	0.26	3.2e-4	-	0.10	0.70	-	1.00	0.00	-	1.00	0.00
		BSS	X	7.4e-300	5.1e5	X	7.4e-300	5.1e8	X	7.4e-300	1.00	X	7.4e-300	1.7e4
TF-Encrypted	2	ASS	-	0.67	1.7e-4	-	0.03	0.50	X	7.4e-300	2.00	-	1.00	0.00
		RSS	X	5.1e-260	9.9e-3	X	7.4e-300	27.31	/	/	/	/	/	/
SecretFlow	2	ASS	X	1.0e-44	6.5e-3	X	7.4e-300	41.70	-	1.00	0.00	-	1.00	0.00
		RSS	X	7.4e-300	9.5e-3	X	1.1e-187	2.68	-	1.00	0.00	-	1.00	0.00

```

def decode(self, tensor):
    """Decodes fixed-point integers to floats by dividing by scale.
    ↪ scale = 2^f, where f is the number of fractional bits."""
    ... # Input validation omitted
    if self._scale > 1:
        correction = (tensor < 0).long()
        # Error: divides by (scale - 1) instead of scale for
        ↪ negatives
        dividend = tensor.div(
            self._scale - correction, rounding_mode="floor"
        )
        remainder = tensor % self._scale
        remainder += (
            (remainder == 0).long() * self._scale * correction
        )
        tensor = (
            dividend.float() + remainder.float() / self._scale
        )
    ... # Other cases omitted
    return tensor.data

```

Figure 4: Negative decoding error in CrypTen’s ASS scheme. The function converts fixed-point encoded integers back to floating-point by dividing by the scale factor (2^f). The erroneous line (highlighted in red) incorrectly uses $\text{scale} - 1$ as the divisor for negative numbers, causing systematic proportional decoding errors.

Impact. This NEV systematically corrupts all negative output values. For applications that produce negative results, such as privacy-preserving financial risk models computing portfolio losses, this vulnerability would lead to systematic bias in loss estimation.

5.3.2 Case Study 2: Modulus Conversion Error (TF-Encrypted-ASS). TF-Encrypted-ASS contains a vulnerability in comparison operations that causes inverted results. For example, the comparison $\text{less}(0, 1)$ should return 1 (true) since $0 < 1$, but incorrectly returns 0 (false). As shown in Table 1, this manifests as $|\Delta| = 2.00$, representing completely inverted Boolean results.

Root Cause. TF-Encrypted-ASS computes comparisons by determining the most significant bit (MSB) of the difference $z = x - y$. The

MSB of z is computed by converting secret shares from modulus 2^{64} to modulus $2^{64} - 1$, and then computing the least significant bit of $2z$ (following the design in SecureNN [31]). As shown in Figure 5, the `_cast_backing_private` function in TF-Encrypted performs the conversion by independently casting each share, which fails to account for modular arithmetic wrap-around. Consider an example where $z = -1$ (represented as $2^{64} - 1$ in modulus 2^{64}) is shared as $\llbracket z \rrbracket_1 = 2^{64} - 50$, $\llbracket z \rrbracket_2 = 49$. The erroneous implementation in TF-Encrypted proceeds as follows:

- (1) $\llbracket y \rrbracket_1 = \llbracket z \rrbracket_1 \bmod (2^{64} - 1) = 2^{64} - 1 - 49$
- (2) $\llbracket y \rrbracket_2 = \llbracket z \rrbracket_2 \bmod (2^{64} - 1) = 49$
- (3) Reconstruct in modulus $2^{64} - 1$:

$$\llbracket y \rrbracket_1 + \llbracket y \rrbracket_2 = (2^{64} - 1 - 49) + 49 = 2^{64} - 1 \equiv 0 \pmod{2^{64} - 1}$$

Because the converted value is 0 instead of -1 , the subsequent MSB computation operates on the wrong value. The protocol expects $\text{MSB}(-1) = 1$ (indicating a negative number), but the erroneous conversion causes it to compute $\text{MSB}(0) = 0$ (indicating zero or a positive number). This inverts the comparison result: $\text{less}(0, 1)$ should return 1 (true, since $0 < 1$), but incorrectly returns 0 (false) due to the conversion error.

Impact. This NEV affects all comparison-based operations, including conditional logic, sorting algorithms, and min/max calculations. For applications that rely on secure comparison, this NEV may lead to incorrect decision-making: for example, in privacy-preserving medical diagnosis, patient risk score comparisons may be reversed, leading to high-risk patients being classified as low-risk and vice versa.

5.3.3 Case Study 3: Input Precision Handling Error (MP-SPDZ-BSS). MP-SPDZ-BSS contains a critical vulnerability where secret-shared inputs are processed with incorrect precision parameters. This manifests as errors in all protocols. For example, when inputting $[-0.95, -1.86, -1.54, 0.68]$ to the MatMult protocol, MP-SPDZ-BSS produces output -515130364.69 instead of the expected -3981.60 with a $129,000\times$ error magnitude that renders BSS completely unusable for fixed-point arithmetic.

```

def _cast_backing_private(prot, x: PondPrivateTensor,
    ↪ backing_dtype):
    """Cast shares to different modulus for MSB computation."""
    x0, x1 = x.unwrapped
    with tf.name_scope("cast_backing"):
        with tf.device(prot.server_0.device_name):
            # Error: Converting shares individually causes
            ↪ wrap-around
            y0 = x0.cast(backing_dtype)
        with tf.device(prot.server_1.device_name):
            y1 = x1.cast(backing_dtype) # Should convert
            ↪ reconstructed value
    return PondPrivateTensor(prot, y0, y1, x.is_scaled)

```

Figure 5: Modulus conversion error in TF-Encrypted. Independent share conversion causes incorrect results when the shared value is -1 due to wrap-around in modular arithmetic.

Root Cause. As shown in Figure 6, the `get_input_from()` method in `Compiler/GC/types.py` incorrectly uses a global precision parameter (`sbitfix.f`) instead of the class-specific precision parameter (`cls.f`) when loading secret-shared inputs. When the class is configured with precision $f = 10$ but the global parameter defaults to $f = 16$, a secret value 3.5 will be incorrectly processed: it should be encoded as integer $3584 = 3.5 * 2^{10}$, but is instead encoded as $229376 = 3.5 * 2^{16}$. This error compounds through subsequent operations, and after 10 secure matrix multiplication operations, produces the observed $129,000\times$ cumulative error.

```

@classmethod
def get_input_from(cls, player, size=1):
    # Error: Uses global precision instead of class-specific
    return cls._new(cls.int_type.get_input_from(
        player, size=size, f=sbitfix.f)) # Should be: cls.f

```

Figure 6: Input precision error in MP-SPDZ. The function uses global precision `sbitfix.f` instead of class-specific `cls.f`, causing inputs to be scaled with incorrect precision.

Impact. This vulnerability affects all operations in the BSS scheme that process fixed-point inputs. The error magnitudes render the scheme completely unusable for fixed-point arithmetic.

5.4 RQ3: Time Efficiency of MPCArbiter

As shown in Table 2, we evaluate the runtime of MPCArbiter across three phases: input generation, NEV detection, and NEV localization.

Overall Efficiency. The time to test an implementation ranges from 161s to 1136s. MPCArbiter runs fully automatically without human intervention and can be scheduled during off-hours. Given that MPCArbiter detects 10 unique NEVs in four widely-used frameworks, the testing time is highly acceptable.

Runtime Breakdown by Phase. The three phases contribute relatively evenly to runtime. Detection averages 204.9s per test, as it requires 500 protocol executions per input to construct output distributions. Input generation averages 163.1s using the evolutionary generation algorithm. Localization averages 211.6s when an NEV

is detected, as it also requires 500 protocol executions to construct intermediate value distributions.

Runtime Variation Across Implementations. Runtime varies dramatically across protocols, schemes, and frameworks due to their differing computation and communication complexities. For example, MP-SPDZ-BSS requires 552s for `MatMult` while MP-SPDZ-ASS needs only 243s, as BSS incurs substantially higher communication overhead than ASS for secure matrix multiplication.

6 Discussion

Infer Which Implementation Contains NEV. While MPCArbiter can effectively detect that at least one of two tested implementations contains an NEV, a natural question arises: how can we infer which specific implementation is buggy without careful manual verification? We discuss two practical approaches to address this challenge: **(1) Distribution comparison against plaintext baseline.** Once MPCArbiter identifies a statistically significant difference between two implementations (indicating at least one contains an NEV), we can then use a plaintext baseline (e.g., a baseline using standard Python float arithmetic) as an additional signal to infer which implementation is more likely buggy. The implementation whose distribution mean deviates more significantly from the plaintext baseline is more likely to contain an NEV. Note that this approach can only be applied after MPCArbiter has detected an NEV in one of the two implementations. If we were to directly compare implementations against the plaintext baseline from the outset, the comparison would be uninformative: due to random truncation noise, both implementations' distribution means will inevitably differ from the baseline, and one will necessarily deviate more than the other. **(2) Multi-implementation comparison.** Another reliable approach is to test against multiple implementations using a majority voting strategy. For instance, if I_A vs. I_B shows a discrepancy, while I_A vs. I_C shows agreement, this suggests that I_B likely contains an NEV.

False Positives and False Negatives. MPCArbiter controls false positives through the significance level α of the KS test. With the default $\alpha = 0.001$, the probability of incorrectly flagging truncation noise as an NEV is at most 0.1%. Users requiring higher confidence can set a smaller α (e.g., $\alpha = 0.0001$). Empirically, all NEVs detected in our evaluation were confirmed with zero false positives observed.

MPCArbiter produces false negatives in two cases: (1) both implementations contain the same error; (2) the test input fails to expose an existing NEV. In the first case, since MPCArbiter relies on differential testing, it detects an NEV only when the two implementations contain different errors; if both implementations happen to contain the same error, their output distributions will coincide. In practice, however, independently developed implementations rarely contain the same error, which is the assumption underlying differential testing in general. In the second case, within a limited testing budget, the input generation may fail to produce an input that exposes an NEV, especially for NEVs that are triggered only by specific inputs.

Future Work. In the future, we plan to: (1) extend MPCArbiter to detecting NEVs in the malicious security setting; (2) apply MPCArbiter to more MPC frameworks (e.g., ABY [8]) to detect and help fix NEVs.

Table 2: Runtime (seconds) of each phase of MPCArbiter. IG: Input Generation, Det: Detection, Loc: Localization. -: No NEV detected (Loc not required); /: Not implemented or cannot run due to other bugs. Overall averages across all valid tests: IG 163.1s, Det 204.9s, Loc 211.6s (when NEV detected).

Framework	Scheme	EleMult				MatMult				Cmp				Sort			
		IG	Det	Loc	Total	IG	Det	Loc	Total	IG	Det	Loc	Total	IG	Det	Loc	Total
CrypTen	ASS	170	241	-	411	173	211	219	603	239	279	286	804	271	345	336	952
	RSS	192	236	-	428	/	/	/	/	305	389	-	694	325	400	411	1136
MP-SPDZ	ASS	102	126	-	228	110	133	-	243	135	166	-	301	230	268	-	498
	RSS	107	143	-	250	109	134	-	243	112	136	-	248	159	178	-	337
	BSS	115	150	141	406	159	180	213	552	107	132	137	376	135	166	180	481
TF-Encrypted	ASS	77	101	-	178	69	92	-	161	113	153	140	406	199	316	-	515
	RSS	168	213	108	489	168	210	107	485	/	/	/	/	/	/	/	/
SecretFlow	ASS	96	118	128	342	225	273	289	787	137	180	-	317	189	240	-	429
	RSS	98	118	133	349	262	341	346	949	153	185	-	338	172	209	-	381

7 Related Work

Differential Testing of Privacy-Preserving Machine Learning.

The works most similar to our proposed MPCArbiter are MPCDIFF [24] and HEDIFF [26], which both apply differential testing to privacy-preserving machine learning (PPML) models. These frameworks compare privacy-hardened machine learning models (using MPC or fully homomorphic encryption) against their plaintext equivalents to identify "deviation inputs"—inputs that cause the PPML model to produce different predictions than the plaintext model. Such deviations arise from design-level tradeoffs inherent to PPML: approximating floating-point numbers with low-precision fixed-point numbers and approximating non-linear activation functions (e.g., Sigmoid) with polynomial functions. By identifying inputs that trigger large deviations, MPCDIFF and HEDIFF can further improve model robustness through precision parameter tuning.

These methods target a fundamentally different problem than MPCArbiter and cannot be applied to detect NEVs. First, MPCDIFF and HEDIFF focus on identifying approximation-induced deviations at the model level to improve robustness, whereas MPCArbiter aims to detect implementation-level NEVs. Second, their testing methodology is fundamentally incompatible with NEV detection for two reasons: (1) they rely on ground truth plaintext baselines for comparison, which do not exist for most MPC protocols that have no plaintext equivalent output due to protocol-specific truncation behavior; (2) even when a plaintext baseline exists, their single-execution comparison approach cannot distinguish between random truncation noise and NEVs, as both cause deviations from the plaintext baseline. In contrast, our proposed MPCArbiter compares two MPC implementations of the same protocol with a distribution-aware oracle, eliminating the need for plaintext baselines and enabling reliable distinction between truncation noise and NEVs.

Formal Verification of MPC Protocols. Formal verification of MPC protocols is typically complex and requires significant manual effort. To increase efficiency, many works focus on computer-aided verification. Most of them work at the protocol level, verifying the protocol logic itself: Haagh et al. [11] utilize the EasyCrypt proof assistant to formalize security proofs against malicious adversaries. Ganther et al. [9] introduce IPDL, a core calculus for providing

an equational logic for proving the observational equivalence of interactive cryptographic protocols. Barthe et al. [4] propose Probabilistic Separation Logic (PSL) to enable formal reasoning about information-theoretic security properties (e.g., uniformity and input independence) in cryptographic constructions. Xie et al. [32] present GAUV, an automated framework based on graph transformations to verify the security of semi-honest MPC protocols. Other works go further to the implementation level, verifying the code that realizes a protocol: Almeida et al. [1] build a verified software stack for secure function evaluation, and Rastogi et al. [27] present Wys^{*}, a DSL for writing MPC programs whose correctness and security properties can be formally verified.

While these works significantly advance the formal verification of MPC, they cannot be used to detect NEVs in existing MPC frameworks. The protocol-level works reason about protocol logic and abstract away the implementation entirely. The implementation-level works do cover code, but only code built within their own toolchains, and thus cannot inspect existing frameworks such as CrypTen or SecretFlow. In contrast, our proposed MPCArbiter is general: it treats an implementation as a black box and requires only two implementations of the same MPC protocol, and thus applies to any existing MPC framework.

Testing of MPC Implementations. Several recent works also focus on testing MPC implementations. For instance, Lin et al. [18] propose MPCGuard, a framework that leverages neural network classifiers to detect data leakage vulnerabilities in MPC implementations. Their method compares the adversary's view in the real world versus an ideal world to identify improper information disclosure. Another line of work focuses on the logical correctness of the compilers that translate high-level programs into MPC protocols. Li et al. [17] apply metamorphic testing to find logical bugs in MPC compilers. This tool automatically mutates high-level MPC programs and checks if the resulting executables are semantically equivalent, successfully uncovering compiler logic errors.

These approaches are crucial, but they target issues orthogonal to those addressed by our proposed MPCArbiter, which focuses on detecting NEVs in MPC implementations.

8 Conclusion

In this paper, we present MPCArbiter, a practical framework for detecting NEVs in MPC implementations. MPCArbiter addresses the fundamental challenge of distinguishing NEVs from truncation noise through a distribution-aware differential oracle, which compares output distributions across multiple MPC implementations with the KS test. To facilitate debugging, we further design an NEV localization method that leverages MPC operation dependency graph to narrow down suspicious code regions. We evaluate MPCArbiter on 36 MPC implementations across four mainstream frameworks (CrypTen, TF-Encrypted, MP-SPDZ, SecretFlow), discovering 10 unique NEVs. Notably, we reveal an NEV in the truncation protocol design that has been widely overlooked by the MPC community. The NEV persists in multiple influential research papers and has propagated into multiple MPC frameworks.

Acknowledgments

This paper is supported by the National Cryptologic Science Fund of China (2025NCSF01010), and the Natural Science Foundation of China (92370120). We thank all anonymous reviewers for their insightful comments.

References

- [1] José Bacerlar Almeida, Manuel Barbosa, Gilles Barthe, François Dupressoir, Benjamin Grégoire, Vincent Laporte, and Vitor Pereira. 2017. A Fast and Verified Software Stack for Secure Function Evaluation. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1989–2006.
- [2] Ethem Alpaydin and Cenik Kaynak. 1998. Optical Recognition of Handwritten Digits Data Set. UCI Machine Learning Repository. <https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>
- [3] Sunpreet Arora, Andrew Beams, Panagiotis Chatzigiannis, Sebastian Meiser, Karan Patel, Srinivasan Raghuraman, Peter Rindal, Harshal Shah, Yizhen Wang, Yuhang Wu, et al. 2024. Privacy-preserving financial anomaly detection via federated learning & multi-party computation. In *2024 Annual Computer Security Applications Conference Workshops*. IEEE, 270–279.
- [4] Gilles Barthe, Justin Hsu, and Kevin Liao. 2019. A probabilistic separation logic. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–30.
- [5] Vance W Berger and YanYan Zhou. 2014. Kolmogorov-Smirnov test: Overview. *Wiley statsref: Statistics reference online* (2014).
- [6] TF-Encrypted Contributors. 2024. TF-Encrypted: A Framework for Privacy-Preserving Machine Learning. <https://github.com/tf-encrypted/tf-encrypted>
- [7] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation* 6, 2 (2002), 182–197. doi:10.1109/4235.996017
- [8] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY-A framework for efficient mixed-protocol secure two-party computation. In *NDSS*.
- [9] Joshua Gancher, Kristina Sojakova, Xiong Fan, Elaine Shi, and Greg Morrisett. 2023. A core calculus for equational proofs of cryptographic protocols. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 866–892.
- [10] Oded Goldreich. 1998. Secure multi-party computation. *Manuscript. Preliminary version* 78, 110 (1998), 1–108.
- [11] Helene Haagh, Aleksandr Karbyshev, Sabine Oechsner, Bas Spitters, and Pierre-Yves Strub. 2018. Computer-aided proofs for multiparty computation with active security. In *Proceedings of the 2018 IEEE 31st Computer Security Foundations Symposium (CSF)*. IEEE, 119–131.
- [12] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and fast secure two-party deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*. 809–826.
- [13] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. doi:10.1145/3372297.3417872
- [14] Brian Knott, Shobha Venkataraman, Awni Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. 2021. CrypTen: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems* 34 (2021), 4961–4973.
- [15] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. 2021. SWIFT: Super-fast and robust privacy-preserving machine learning. In *30th USENIX Security Symposium (USENIX Security 21)*. 2651–2668.
- [16] Dong Li, Xiaofeng Liao, Tao Xiang, Jiahui Wu, and Junqing Le. 2020. Privacy-preserving self-serviced medical diagnosis scheme based on secure multi-party computation. *Computers & Security* 90 (2020), 101701.
- [17] Yichen Li, Dongwei Xiao, Zhibo Liu, Qi Pang, and Shuai Wang. 2024. Metamorphic testing of secure multi-party computation (mpc) compilers. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1216–1237.
- [18] Guopeng Lin, Xiaoning Du, Lushan Song, Weili Han, Jin Tan, Junming Ma, Wenjing Fang, and Lei Wang. 2025. Is MPC Secure? Leveraging Neural Network Classifiers to Detect Data Leakage Vulnerabilities in MPC Implementations. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2809–2826.
- [19] Guopeng Lin, Weili Han, Wenqiang Ruan, Ruisheng Zhou, Lushan Song, Bing-shuai Li, and Yunfeng Shao. 2024. Ents: an efficient three-party training framework for decision trees by communication optimization. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 4376–4390.
- [20] Guopeng Lin, Ruisheng Zhou, Shuyu Chen, Weili Han, Jin Tan, Wenjing Fang, Lei Wang, and Tao Wei. 2025. Kona: An Efficient Privacy-Preservation Framework for KNN Classification by Communication Optimization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*, Vol. 267. PMLR, 37996–38012.
- [21] William M McKeeman. 1998. Differential testing for software. *Digital Technical Journal* 10, 1 (1998), 100–107.
- [22] Payman Mohassel and Peter Rindal. 2018. ABY3: A mixed protocol framework for machine learning. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 35–52.
- [23] Payman Mohassel and Yupeng Zhang. 2017. Secureml: A system for scalable privacy-preserving machine learning. In *2017 IEEE symposium on security and privacy (SP)*. IEEE, 19–38.
- [24] Qi Pang, Yuanyuan Yuan, and Shuai Wang. 2024. MPCDiff: Testing and Repairing MPC-Hardened Deep Learning Models. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium*.
- [25] Arpita Patra and Ajith Suresh. 2020. BLAZE: blazing fast privacy-preserving machine learning. *arXiv preprint arXiv:2005.09042* (2020).
- [26] Yiteng Peng, Daoyuan Wu, Zhibo Liu, Dongwei Xiao, Zhenlan Ji, Juergen Rahmel, and Shuai Wang. 2025. Testing and understanding deviation behaviors in the-hardened machine learning models. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2251–2263.
- [27] Aseem Rastogi, Nikhil Swamy, and Michael Hicks. 2019. Wys*: a DSL for verified secure multi-party computations. In *Proceedings of the International Conference on Principles of Security and Trust*. 99–122.
- [28] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. CryptFlow2: Practical 2-party secure inference. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 325–342.
- [29] SecretFlow Team. 2024. SecretFlow: A Unified Framework for Privacy-Preserving Data Analysis and Machine Learning. <https://github.com/secretflow/secretflow>.
- [30] Paul Voigt and Axel Von dem Bussche. 2017. The eu general data protection regulation (gdpr). *A Practical Guide, 1st Ed., Cham: Springer International Publishing* (2017).
- [31] Sameer Wagh, Divya Gupta, and Nishanth Chandran. 2019. SecureNN: 3-Party Secure Computation for Neural Network Training. *Proceedings on Privacy Enhancing Technologies* 2019, 3 (2019), 26–49.
- [32] Xingyu Xie, Yifei Li, Wei Zhang, Tuowei Wang, Shizhen Xu, Jun Zhu, and Yifan Song. 2024. GAuV: A graph-based automated verification framework for perfect semi-honest security of multiparty computation protocols. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 484–502.
- [33] Andrew C Yao. 1982. Protocols for secure computations. In *23rd Annual Symposium on Foundations of Computer Science*. IEEE, 160–164.

A Ethical Considerations

Our research involves the discovery and responsible disclosure of numeric error vulnerabilities (NEVs) in four MPC frameworks (SecretFlow, CrypTen, TF-Encrypted, MP-SPDZ). We have carefully considered the ethical implications of our work through a stakeholder-based analysis.

Stakeholders, Impacts, and Mitigation. The primary stakeholders include: (1) Framework developers, who may face potential reputational harm from disclosed vulnerabilities but benefit from improving their systems’ reliability; and (2) Framework users, who benefit from more reliable MPC frameworks. To minimize potential harm to developers, we followed responsible disclosure practices by

privately reporting all discovered vulnerabilities with detailed technical descriptions before public disclosure. As of the time of writing, the developers of MP-SPDZ and SecretFlow have confirmed and patched all the NEVs. For frameworks no longer actively maintained (CrypTen) or minimally maintained (TF-Encrypted), we made best-effort attempts to contact developers and document the vulnerabilities for community awareness. For framework users, our work provides benefits by identifying NEVs that could compromise the correctness of their privacy-preserving applications.

Decision to Publish. After weighing the potential benefits against the potential harms, we determined that publication is warranted. The primary benefits include: (1) alerting the research community to a widespread design-level NEV in truncation protocols that exists across multiple influential research papers and MPC frameworks; (2) providing a systematic detection framework (MPCArbiter) that enables developers and users to identify and prevent NEVs in MPC frameworks. The potential harms—primarily reputational concerns for framework developers—have been substantially mitigated through our responsible disclosure practices and coordination with developers. Given this analysis, the benefits of publication substantially outweigh the residual risks. Our work aligns with the principle of beneficence by improving the reliability of MPC implementations for all users and developers.

B Open Science

To support transparency and reproducibility, we provide the following artifacts:

Artifacts. Our artifacts include:

- **MPCArbiter Framework:** Complete implementation of our detection framework, including the distribution-aware differential oracle, input generation strategies, and NEV localization components.
- **Tested MPC Frameworks:** Specific versions of the four evaluated frameworks (SecretFlow, CrypTen, TF-Encrypted, MP-SPDZ) used in our experiments.
- **Test Protocols:** Implementation of the four test protocols (EleMult, MatMult, Cmp, Sort) used in our evaluation.
- **Experimental Scripts:** Scripts to reproduce all experiments presented in Figure 1 and Section 5 (including vulnerability detection results and runtime measurements).

Availability. All artifacts are available at <https://github.com/FudanMPL/MPCArbiter>.

C Tested Protocol Pseudocode

This appendix provides pseudocode for the four MPC operations tested in our evaluation. The pseudocode describes the computational logic that is implemented across all tested frameworks (CrypTen, TF-Encrypted, MP-SPDZ, SecretFlow).

EleMult: As shown in Figure 7, this protocol securely computes the product of 10 secret-shared values. Each secure multiplication operation requires truncation to maintain fixed-point precision.

MatMult: As shown in Figure 8, this protocol performs 10 iterations of secure matrix multiplication operations, which are fundamental in neural network computations.

```
Input: 10 secret-shared values x0, x1, ..., x9
result = 1
for i = 0 to 9 do
    result = Truncate(result * xi)
end for
Output: result
```

Figure 7: EleMult Pseudocode

```
Input: 2x2 secret-shared matrix M
X0 = M
X1 = Transpose(X0)
for i = 1 to 10 do
    X0 = Truncate(X0 * X1) # Secure matrix multiplication
end for
result = Sum of all elements in X0
Output: result
```

Figure 8: MatMult Pseudocode

Cmp: As shown in Figure 9, this protocol tests nested secure comparison operations where boolean results are used in subsequent comparisons.

```
Input: two secret-shared values a and b
x0 = (b < a)
x1 = (b <= a)
x2 = (a > b)
x3 = (a >= b)
x4 = (a == b)
x5 = (a != b)
x6 = (x0 > x3)
x7 = ((x1 == x4) != (x2 < x5))
x8 = ((x0 + x4) >= (x3 + x5))
result = x6 + x7 + x8
Output: result
```

Figure 9: Cmp Pseudocode

Sort: As shown in Figure 10, this protocol sorts an array of secret-shared values.

```
Input: array of 5 secret-shared values A = [a[0], a[1], a[2], a[3], a[4]]
A = sort(A)
Output: sorted array A
```

Figure 10: Sort Pseudocode

D Detailed Analysis of Truncation Bias

In this section, we provide the detailed mathematical derivation showing the fundamental difference between symmetric and asymmetric truncation protocols in terms of their expected absolute deviation from the exact real-number result.

Problem Setup. Let $z = \llbracket z \rrbracket_1 + \llbracket z \rrbracket_2$ be a secret-shared integer value with $2f$ fractional bits, representing the real number $T_{real} = z/2^{2f}$. The truncation operation converts this to an integer with f fractional bits. We assume the fractional part $c = z \bmod 2^f$ follows a uniform distribution over $\{0, 1, \dots, 2^f - 1\}$.

Let $r_1 = \lfloor z \rfloor_1 \bmod 2^f$ and $r_2 = \lfloor z \rfloor_2 \bmod 2^f$ denote the fractional parts of the two shares. Note that $(r_1 + r_2) \bmod 2^f = c = z \bmod 2^f$. Since the shares are uniformly random, for a fixed c the value r_1 is uniformly distributed over $\{0, 1, \dots, 2^f - 1\}$, which determines r_2 accordingly. Throughout this section, $\mathbb{E}_c[\cdot]$ denotes the expectation taken over the randomness of r_1 for a fixed c .

D.1 Symmetric Truncation Analysis

For symmetric truncation (the erroneous approach), both parties compute floor:

$$T_B = \lfloor \lfloor z \rfloor_1 / 2^f \rfloor + \lfloor \lfloor z \rfloor_2 / 2^f \rfloor$$

Key Observation: The result T_B depends on whether there is a “carry” when adding the fractional parts of the two shares.

Case 1: No carry ($r_1 + r_2 < 2^f$). This occurs when $r_1 \leq c$.

- Truncation result:

$$T_B = \lfloor z / 2^f \rfloor$$

- Probability: $P(r_1 \leq c) = \frac{c+1}{2^f}$

Case 2: Carry occurs ($r_1 + r_2 \geq 2^f$). This occurs when $r_1 > c$.

- Truncation result:

$$T_B = \lfloor z / 2^f \rfloor - 1$$

- Probability: $P(r_1 > c) = \frac{2^f - c - 1}{2^f}$

Expected Absolute Deviation. The real-number representation of the original value is $T_{real} = z / 2^{2f}$. Note that $z = 2^f \lfloor z / 2^f \rfloor + c$, so:

$$T_{real} = \frac{2^f \lfloor z / 2^f \rfloor + c}{2^{2f}} = \frac{\lfloor z / 2^f \rfloor}{2^f} + \frac{c}{2^{2f}}$$

For each case:

- When $T_B = \lfloor z / 2^f \rfloor$ (no carry):

$$|T_B / 2^f - T_{real}| = \left| \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{c}{2^{2f}} \right| = \frac{c}{2^{2f}}$$

- When $T_B = \lfloor z / 2^f \rfloor - 1$ (carry occurs):

$$|T_B / 2^f - T_{real}| = \left| \frac{\lfloor z / 2^f \rfloor - 1}{2^f} - \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{c}{2^{2f}} \right| = \frac{2^f + c}{2^{2f}}$$

For a fixed c , the expected absolute deviation is:

$$\mathbb{E}_c[|T_B / 2^f - T_{real}|] = \frac{c+1}{2^f} \cdot \frac{c}{2^{2f}} + \frac{2^f - c - 1}{2^f} \cdot \frac{2^f + c}{2^{2f}} = \frac{2^f - 1}{2^{2f}}$$

Note that this result is *independent of c*. Averaging over all $c \in \{0, 1, \dots, 2^f - 1\}$:

$$\mathbb{E}[|T_B / 2^f - T_{real}|] = \frac{2^f - 1}{2^{2f}} \approx \frac{1}{2^f}$$

D.2 Asymmetric Truncation Analysis

For asymmetric truncation (the correct approach), P_1 computes floor and P_2 computes ceiling:

$$T_C = \lfloor \lfloor z \rfloor_1 / 2^f \rfloor + \lceil \lceil z \rceil_2 / 2^f \rceil$$

Key Observation: Both truncation methods produce results that depend on the random sharing. The difference lies in which two outcomes occur and with what probabilities: under symmetric truncation the lost carry is never compensated, whereas the ceiling

operation compensates for it whenever $r_2 \neq 0$, yielding a lower expected deviation.

Case 1: No carry ($r_1 + r_2 < 2^f$). This occurs when $r_1 \leq c$.

- For $r_1 < c$ (which occurs with probability $c/2^f$), we have $r_2 = c - r_1 > 0$, so $\lceil \lfloor z \rfloor_2 / 2^f \rceil = \lfloor \lfloor z \rfloor_2 / 2^f \rfloor + 1$ and

$$T_C = \lfloor z / 2^f \rfloor + 1$$

- For $r_1 = c$ (which occurs with probability $1/2^f$), we have $r_2 = 0$, so the ceiling coincides with the floor and

$$T_C = \lfloor z / 2^f \rfloor$$

Case 2: Carry occurs ($r_1 + r_2 \geq 2^f$). This occurs when $r_1 > c$.

- In this case, $r_2 = c - r_1 + 2^f$ (due to modular arithmetic), so $r_2 > 0$ and hence $\lceil \lfloor z \rfloor_2 / 2^f \rceil = \lfloor \lfloor z \rfloor_2 / 2^f \rfloor + 1$
- This extra +1 compensates exactly for the carry lost by the two independent shifts, giving

$$T_C = \lfloor z / 2^f \rfloor$$

- Probability: $P(r_1 > c) = \frac{2^f - c - 1}{2^f}$

Simplified Probability Analysis. Combining the two sub-cases that yield $\lfloor z / 2^f \rfloor$ (namely $r_1 = c$ in Case 1 and the carry case), the outcome probabilities are:

- $P(T_C = \lfloor z / 2^f \rfloor + 1) = \frac{c}{2^f}$ (occurs when $r_1 < c$)
- $P(T_C = \lfloor z / 2^f \rfloor) = \frac{2^f - c}{2^f}$ (occurs when $r_1 \geq c$)

Expected Absolute Deviation. For each case:

- When $T_C = \lfloor z / 2^f \rfloor + 1$:

$$|T_C / 2^f - T_{real}| = \left| \frac{\lfloor z / 2^f \rfloor + 1}{2^f} - \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{c}{2^{2f}} \right| = \frac{2^f - c}{2^{2f}}$$

- When $T_C = \lfloor z / 2^f \rfloor$:

$$|T_C / 2^f - T_{real}| = \left| \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{\lfloor z / 2^f \rfloor}{2^f} - \frac{c}{2^{2f}} \right| = \frac{c}{2^{2f}}$$

For a fixed c , the expected absolute deviation is:

$$\mathbb{E}_c[|T_C / 2^f - T_{real}|] = \frac{c}{2^f} \cdot \frac{2^f - c}{2^{2f}} + \frac{2^f - c}{2^f} \cdot \frac{c}{2^{2f}} = \frac{2c(2^f - c)}{2^{3f}}$$

Averaging over all $c \in \{0, 1, \dots, 2^f - 1\}$:

$$\mathbb{E}[|T_C / 2^f - T_{real}|] = \frac{1}{2^f} \sum_{c=0}^{2^f-1} \frac{2c(2^f - c)}{2^{3f}} = \frac{2^f - 1}{3 \cdot 2^{3f}} \approx \frac{1}{3 \cdot 2^f}$$

D.3 Comparison and Conclusion

The expected absolute deviations are:

- **Symmetric truncation (Erroneous):** $\mathbb{E}[|T_B / 2^f - T_{real}|] = \frac{2^f - 1}{2^{2f}} \approx \frac{1}{2^f}$
- **Asymmetric truncation (Correct):** $\mathbb{E}[|T_C / 2^f - T_{real}|] = \frac{2^f - 1}{3 \cdot 2^{3f}} \approx \frac{1}{3 \cdot 2^f}$

Symmetric truncation has approximately 3× larger expected absolute deviation compared to asymmetric truncation.